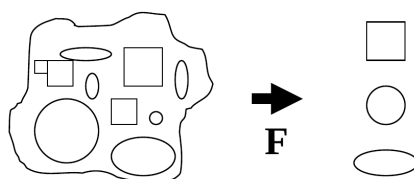


Backpropagation Feed-Forward Neural Networks

Vasili Gavrilov

Seminar in Artificial Intelligence (1997)



Preface (2026)

I wrote this in 1997 as a seminar thesis in neural networks, the work required to complete my undergraduate computer science degree. The task my professor gave me was to derive the delta-rule for multi-layer feed-forward networks in its general form: the prior work by Hinton and collaborators had established backpropagation at the perceptron level, and the seminar's job was to push the derivation through to the general multi-layer case. I worked it out in tensor form, and added a small meta-analysis of the existing NN literature of the time. In retrospect, amusing how few papers were available to survey. The illustrations are vintage MS Paint, drawn in a rush during the summer before I left for Canada.

For twenty-nine years it sat in a folder. The decision to publish came from an unexpected source: a long conversation with a large language model about a separate topic. At some point the model said the report might still be useful as a beginner's introduction to backpropagation feed-forward networks, because it is short, self-contained, math-first, builds from first principles, and old enough that nothing in it has the polish of a modern textbook.

That last bit is what made me pause. Most current tutorials assume you already know what a gradient is, drop you into PyTorch on page two, and treat backprop as a black box behind `loss.backward()`. This report does the opposite. It starts at the biological neuron, builds up the math, derives backpropagation from gradient descent and the chain rule, and ends with two real applications: cell classification for cancer diagnosis, and the LeCun handwritten-digit work that, in retrospect, is the origin story of convolutional networks.

For a beginner today the math is unchanged since 1986, and a reader with calculus (through partial derivatives) and basic linear algebra can follow the derivation cover-to-cover in one sitting. What is missing is mostly cosmetic: ReLU instead of sigmoid for activation, Adam instead of fixed-rate gradient descent, and the post-2012 deep-learning

toolkit. None of that changes the foundations the report covers; you layer the modern material on top once the foundations are clear.

Historical Note (2026). This document is a digitized archival release of a seminar coursework report originally completed in 1997 at the Open University of Israel under the supervision of Professor Ehud Gudes. The substance is preserved as a snapshot of early neural network methodology. Corrections and restorations made during the 2026 digitization pass: a few grammatical fixes; one tensor index aligned with the row-receives, column-sends convention used throughout; a weight-initialization range restored from a transcription error ($2.4 f_i \rightarrow 2.4/f_i$); an author name order corrected (Ciamac Moallemi); the Rumelhart, Hinton, Williams 1986 backpropagation paper restored to the bibliography after being lost in transcription; several sections and fourteen of the original 1997 illustrations dropped by an earlier rewrite have been restored from the source PDF; and a duplicate “meta-analysis” applications section introduced by that rewrite has been removed. This is Ver.2 of the restoration: several additional pictures from the original that had been lost have since been found in the archive and reinserted.

Abstract

This coursework concerns backpropagation feed-forward neural networks (BPFFNNs), which historically comprised a large fraction of successful neural-network applications. The report reviews the main architectural ideas behind feed-forward networks, introduces the standard neuron model, and derives the generalized delta rule (backpropagation) as gradient descent in weight space, including a consistent tensor (matrix) form of the update. Two practical applications are discussed: classifying cells for cancer diagnosis and handwritten digit recognition. Finally, the report argues that incorporating problem-specific heuristics into the network topology and preprocessing is often necessary to obtain a network that is trainable, convergent, and able to generalize without severe underfitting or overfitting.

Contents

1	Introduction to Neural Networks	5
1.1	Differences from the von Neumann model	5
1.2	Comparison: brain vs. computer (1997 estimates)	6
1.3	Hardware perspective (Flynn's classification)	6
2	Neurons	6
2.1	The basic neuron model	7
2.2	Why nonlinearity is essential	7
2.3	Bias as an extra input	8
2.4	Example: logic gates	9
3	Feed-Forward Networks	9
3.1	Layered structure	9
3.2	Supervised and unsupervised learning	10
3.3	Function approximation (informal)	10
4	Network Topology	10
4.1	Topology and inductive bias	10
5	Learning	11
5.1	Every node is a classifier	11
5.2	XOR and the limits of one perceptron	12
5.3	Training, test, and validation sets	12
5.4	Underfitting, overfitting, and the spline analogy	13
5.5	The perceptron as a starting point	14
5.6	Multi-layer feed-forward networks	14
6	Backpropagation	14
6.1	Error function	15
6.2	Gradient descent in weight space	15
6.3	Derivation of the generalized delta rule	16
6.4	Local error signal (delta / beta)	16
6.5	Sigmoid-specific form	17
6.6	Tensor (matrix) form	17
6.7	Backpropagation algorithm (outline)	17

6.8	Learning rate and momentum	17
7	Applications	18
7.1	Classifying cells for cancer diagnosis	18
7.1.1	Data and processing pipeline	19
7.1.2	Results	21
7.2	Handwritten digit recognition using constrained automatic learning . . .	21
7.2.1	Preprocessing	21
7.2.2	Shared weights and feature maps	22
7.2.3	Network architecture and training	22
7.2.4	Performance and impact	23
8	Summary	23
9	Bibliography	24

1 Introduction to Neural Networks

In this report, *neural networks* (NN) refers to *artificial neural networks* (ANNs): mathematical models inspired by biological neural systems. Historically, the original goal of ANNs was to model and simulate biological neural computation. In modern usage, however, “neural network” is frequently used for engineered systems whose architecture and operation may differ substantially from biology.

Although there are many definitions, most share a common core: a neural network is a massively parallel, distributed processor (typically without an overall system clock) that can learn from examples and often exhibits some capability for generalization beyond the training data. A neural network contains many simple interconnected processing elements with local memory rather than a small number of complex processors. Each element operates primarily on local information and can operate asynchronously.

In a broad sense, neural networks can be viewed as a non-von Neumann approach to computation, where many competing hypotheses are processed simultaneously. Implementations range from software simulators to dedicated hardware accelerators.

1.1 Differences from the von Neumann model

In a von Neumann computer the programmer must know in advance the exact sequence of steps the algorithm performs. The advantage of NN is the absence of that requirement: it is sufficient to supply the network with enough well-chosen examples of the desired input-output behavior, which can be considered as “teaching” the network. Selecting a good training set may take more time and effort than the training itself. A NN cannot predict a non-existent function, but it can approximate any mapping between input and output spaces (any computable function) given enough resources and training data.

The main features distinguishing a NN from a traditional digital computer are:

- Signal processing (rather than symbol processing).
- Information stored in weights (rather than in a program).
- Robustness in the presence of noise: small input perturbations do not collapse the output.
- Robustness to partial hardware failure: damage affects only a subset of outputs rather than disabling the system.
- High-level concepts distributed across the whole network rather than localized in one memory cell.
- Generalization: the ability to deal with patterns not seen during training.

It is worth noting that NN are not more powerful in the formal sense than ordinary digital computers (they compute the same class of computable functions), but they make some tasks much easier than algorithmic programming does. Telling photographs of men from photographs of women, for instance, has no clean rule-based algorithm; yet a five-year-old performs it reliably, even when the woman has short hair. NN are well-suited to that kind of task.

1.2 Comparison: brain vs. computer (1997 estimates)

The biological inspiration is concrete in scale terms. [2026: *The orders-of-magnitude comparison below is roughly what the field reported in 1997; modern figures look very different.*]

	Computer	Brain
Computational units	10^5 gates	10^{11} neurons
Storage units	10^9 RAM, 10^{10} disk	10^{11} synapses
Cycle time	10^{-8} seconds	10^{-3} seconds
Bandwidth	10^9 bits/sec	10^{14} bits/sec
Neuron updates per sec	10^5	10^{14}

An ANN simulator running on a von Neumann host needs roughly 10^2 cycles to model a single neuron firing, while a brain performs all neuron updates simultaneously. The computer is about 10^6 times faster at raw switching, yet a brain recognizes a face in under a second. The difference is massive parallelism. (Dedicated NN hardware or clusters narrow this gap; the comparison above assumes ANN simulated on a serial machine.) [2026: *This was already true in 1997 with the AT&T ANNA chip and Connection Machines; it is overwhelmingly true today with GPUs.*]

1.3 Hardware perspective (Flynn’s classification)

Flynn’s classification organizes computer architectures along two axes: how many instruction streams the machine executes at once, and how many data streams it operates on at once. The four resulting quadrants are:

	Single Data Stream	Multiple Data Streams
Single Instruction Stream	SISD (classical uniprocessor)	SIMD (vector / array processor)
Multiple Instruction Streams	MISD (rare; systolic array)	MIMD (multiprocessor, cluster)

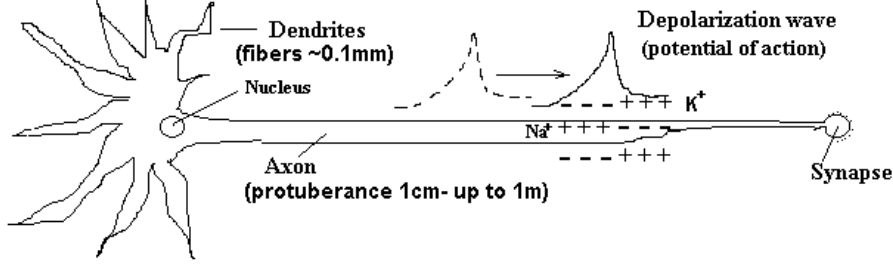
Classical uniprocessors fall under SISD. Vector and array supercomputers (the Cray series, ILLIAC IV) are the canonical SIMD machines, applying one instruction to many data elements at once. Large multiprocessor systems and transputer-based hardware are MIMD. The MISD quadrant has few practical instances; systolic arrays are sometimes placed there.

Neural networks fit naturally into the MIMD model. Each neuron is conceptually an independent processor operating on its own inputs and weights. In practice, many neural computations (e.g., applying the same activation function across all units in a layer, or computing layer-wise vector–matrix products) are regular enough that SIMD hardware also accelerates them effectively; specialized chips for backpropagation networks of this period typically exploited that regularity.

2 Neurons

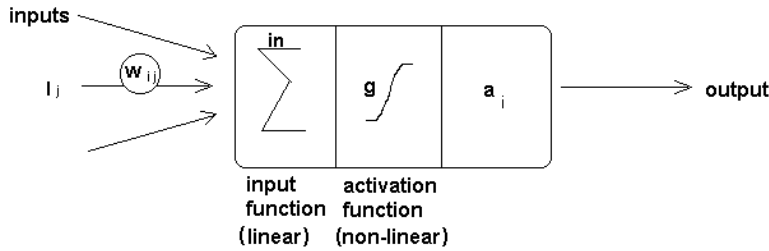
Since NN are based on the architecture of animal brains, we begin with a brief description of a real neuron. A neuron has many dendrites and (usually) one long axon; the axon

contacts the dendrites of other cells through synapses. Treating the dendrites of a single cell as its inputs, the cell does nothing until the collective influence of all inputs (roughly 10^5 per neuron) reaches the firing threshold. Knowledge is stored at the synapses as connection strengths (amounts of various chemical mediators), which we model as *synaptic weights*. When the cell fires, a depolarization wave (a wave-like local Na^+/K^+ polarization) travels down the axon and reaches the synapses at its end, where it influences the next neuron in turn.



2.1 The basic neuron model

The computing units of an ANN are called *artificial neurons*, or *threshold logic units* (TLU). The basic schema is:



Each incoming link carries an associated weight (a scalar parameter that determines how strongly one node influences another). A positive weight ($w > 0$) is called an *excitation*; a negative weight ($w < 0$) is called an *inhibition*. Weights are obtained from learning. For inputs $I_1, \dots, I_n$, weights $w_1, \dots, w_n$, and a threshold θ , define the pre-activation

$$in_i = \sum_{j=1}^n w_{ij} I_j, \quad (1)$$

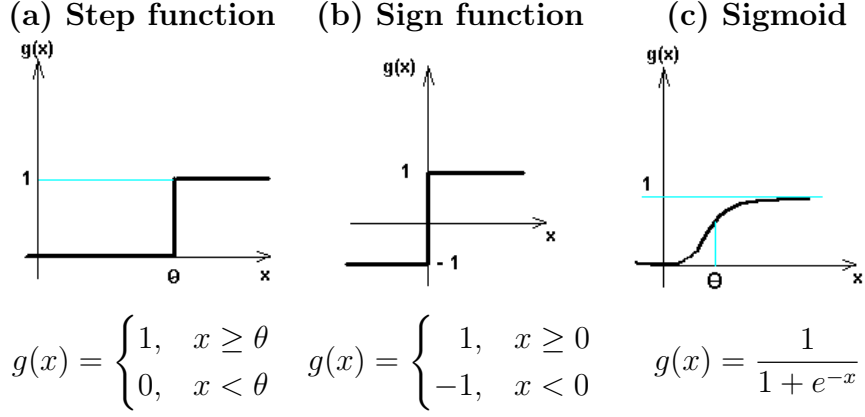
and the output

$$a_i = g(in_i), \quad (2)$$

where $g(\cdot)$ is the activation function. The neuron activates when in_i reaches the threshold θ ; in vector form this condition is $\mathbf{w} \cdot \mathbf{I} \geq \theta$.

2.2 Why nonlinearity is essential

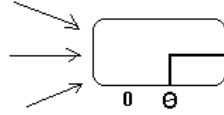
Note that g cannot be linear; if it were, a multi-layer network would have the same expressive power as a single linear layer (since the sum of linear functions is linear). Nonlinear activation is therefore essential for representing non-linear decision boundaries and for approximating complex functions. Different methods may use different choices of g ; the requirement is non-linearity. The three common choices are:



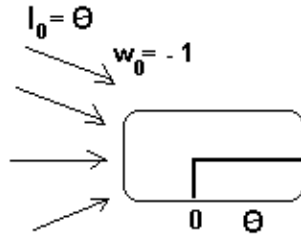
The sigmoid is the most interesting choice for the purposes of this report: it is differentiable around the threshold θ (the chain rule below requires this). Other nonlinear functions also work in practice; an interesting example using $\sin(x)$ as activation is mentioned later. [2026: Sigmoid was the 1997 default for hidden layers. Today the default is ReLU, $g(x) = \max(0, x)$, which removes the vanishing-gradient problem of deep sigmoid networks and trains much faster. Sigmoid is still used at output units for binary classification, and softmax for multi-class output.]

2.3 Bias as an extra input

A TLU with a separate threshold θ can be drawn as:



It is often more convenient to fold the threshold into the weights themselves (so that learning adjusts weights only, not weights and a separate threshold) by introducing an extra input I_0 :



With $I_{0i} = \theta$ and $w_{0i} = -1$, the threshold becomes

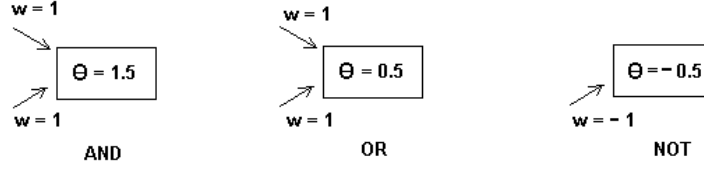
$$w_{0i}I_{0i} = -\theta, \quad (3)$$

and the activation condition collapses neatly:

$$a_i = g(in_i) = g\left(\sum_{j=1}^n w_{ij}I_{ij}\right) = g\left(\sum_{j=0}^n w_{ij}I_{ij}\right). \quad (4)$$

2.4 Example: logic gates

A single TLU with appropriate weights and a step activation implements any one of the basic Boolean functions. The classic three:

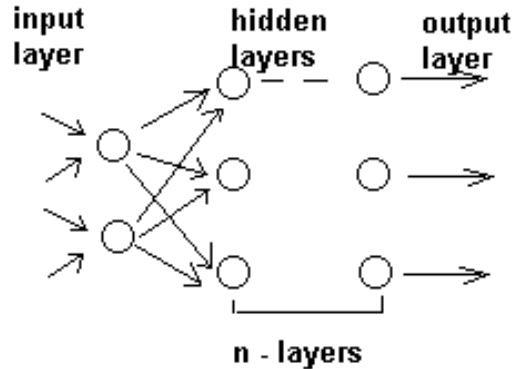


AND: $w_1 = w_2 = 1$, $\theta = 1.5$. OR: $w_1 = w_2 = 1$, $\theta = 0.5$. NOT: $w = -1$, $\theta = -0.5$. Since AND, OR, and NOT form a functionally complete set, any Boolean function can in principle be assembled from such elements.

3 Feed-Forward Networks

3.1 Layered structure

A feed-forward network is organized into layers: an input layer, one or more hidden layers, and an output layer. Information flows from inputs to outputs in a strictly feed-forward way. This is in contrast to recurrent networks, which contain feedback connections (Hopfield networks are a classical example).



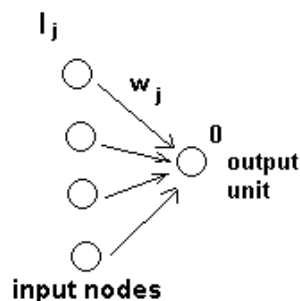
Let $a^{(\ell)}$ be the vector of activations in layer ℓ . The feed-forward computation for layer ℓ is

$$z^{(\ell)} = W^{(\ell)}a^{(\ell-1)} + b^{(\ell)}, \quad (5)$$

$$a^{(\ell)} = g(z^{(\ell)}), \quad (6)$$

where $W^{(\ell)}$ is the weight matrix and $b^{(\ell)}$ is the bias vector.

The first studied network of this kind (1950s) was the *perceptron*: a single-layer network with no hidden units. It is the most trivial network architecture. The limitations of perceptrons are revisited in the Learning section.



A single-layer perceptron: input nodes I_j feed into one output unit through weights w_j , with no intermediate layer.

3.2 Supervised and unsupervised learning

A common distinction is between supervised and unsupervised learning networks. In supervised learning, an external “teacher” (supervisor) specifies the desired behavior during training. In unsupervised learning, the network self-organizes structure from the data (Kohonen networks are a standard example). Some networks do not learn at all (they compute a fixed function).

3.3 Function approximation (informal)

Feed-forward networks with at least one sufficiently large hidden layer can approximate a broad class of continuous functions on compact domains. In practice, approximation quality depends on the number of hidden units, the amount and quality of training data, and the effectiveness of the training algorithm.

There is no simple closed form for the “ideal” network architecture: network design is an art. The right architecture depends on the number of training cases, the noise level, and the complexity of the function being learned. Some problems with one input and one output need thousands of hidden units; some problems with thousands of inputs need almost none. In the sections that follow, all architectures are designed by hand rather than discovered automatically.

4 Network Topology

4.1 Topology and inductive bias

Network topology is the pattern of connections among units. It determines how information propagates and, crucially, encodes an *inductive bias*: constraints that reduce the hypothesis space so learning is feasible with limited data.

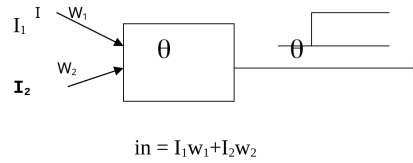
In application work, topology is rarely chosen arbitrarily. It is usually adapted to the structure of the input domain (e.g., 2D geometry for images) and to invariances required for generalization (e.g., shift invariance in character recognition).

5 Learning

Learning (teaching, training) a network is the process of tuning its parameters to fit a training set. Many learning methods exist and new ones appear constantly; from a statistician's point of view a neural network performs nonlinear regression. The initial network has randomly assigned weights (zeros or small numbers close to zero). The network is updated to make it consistent with a sequence of input–output examples: small adjustments to weights reduce the difference between observed and predicted outputs. Updates are grouped into *epochs* (one pass over all training examples). The process repeats until the error becomes negligibly small.

5.1 Every node is a classifier

Every node in the network behaves as a classifier, partitioning its input space into two regions. Consider a single perceptron node with two inputs (dropping the node index for clarity):



Two-input perceptron with explicit inputs I_1, I_2 , weights w_1, w_2 , threshold θ , and a step activation; the unit fires when the linear sum reaches θ .

The activation condition $\mathbf{w} \cdot \mathbf{I} \geq \theta$ reads in components as

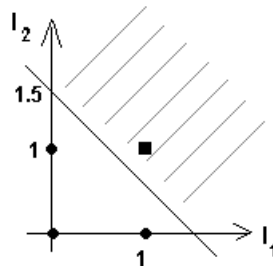
$$w_1 I_1 + w_2 I_2 \geq \theta. \quad (7)$$

Since this is a linear inequality, it corresponds to a half-plane in (I_1, I_2) space separated by the line

$$I_2 = -\frac{w_1}{w_2} I_1 + \frac{\theta}{w_2}. \quad (8)$$

In general, with n inputs the boundary is an n -dimensional hyperplane embedded in $(n + 1)$ -dimensional space.

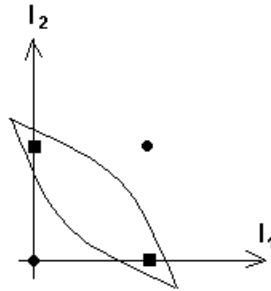
Worked example. Let $w_1 = w_2 = 1$ and $\theta = 1.5$. Substituting into $w_1 I_1 + w_2 I_2 = \theta$ gives $I_1 + I_2 = 1.5$. If inputs are restricted to $\{0, 1\}$, the solutions partition the unit square:



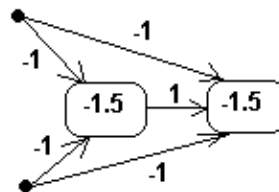
Learning the perceptron then reduces to adjusting w and θ until the line separates the two classes.

5.2 XOR and the limits of one perceptron

Consider two regions in (I_1, I_2) space where one is enclosed by the other; no single straight line separates them.



The Boolean XOR function is the classical example of a linearly non-separable function. Rosenblatt's perceptron-convergence theorem (first proved in 1960) guarantees that the perceptron learning rule converges to a correct set of weights, provided the target function is linearly separable; in particular, a perceptron learns successfully only from noise-free, linearly-separable data. For functions like XOR the only remedy is to add a hidden unit:

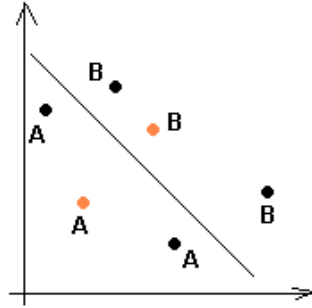


5.3 Training, test, and validation sets

The standard terminology (following Ripley) is:

- **Training set:** examples used for learning.
- **Test set:** examples used to evaluate performance.
- **Validation set:** examples used to tune classifier parameters (network topology, threshold function, etc.).
- **Generalization:** the property of the network to make correct predictions for cases that were not in the training set.

Consider the picture below: black dots are training examples, lighter ones are test examples. Even though the lighter dots were never used to fit the weights, they are still classified correctly: A as A , B as B .

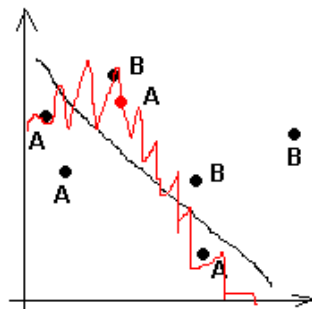


Generalization is the most critical issue in network design. In statistical terms it is the difference between interpolation (predicting within the range of training data) and extrapolation (predicting outside it). The main aim is to make the network rely on interpolation by collecting enough good training data, usually the hardest part of the project.

5.4 Underfitting, overfitting, and the spline analogy

A network that lacks enough internal degrees of freedom to model the target function will fail to fit even the training set. This is *underfitting*: the network is too trivial, rigid, or low-capacity. The other extreme is *overfitting*: the network has so many parameters that it can memorize the training data (including noise) without learning the underlying regularity.

A useful analogy is spline interpolation. To pass a curve through a set of pins, one could choose a flexible rope: it will pass through every pin perfectly, but the curve in between is essentially arbitrary and the rope has not learned anything generalizable about the data; it has memorized it. That is overfitting. At the opposite extreme, a stiff metal rod cannot bend enough to reach the pins at all; it captures only the simplest linear trend and misses everything else. That is underfitting. The ideal classifier is flexible enough to fit the data but rigid enough to derive the underlying knowledge rather than enumerate the inputs.



Overfitting and underfitting.

A practical rule of thumb is that the number of trainable weights influencing any output should be smaller than the number of training samples. Increasing the number of weights can speed convergence but increases the risk of overfitting; a very small network may not learn the desired mapping at all. A related concern with too-complex networks is slow convergence: even when they eventually learn, training can be impractically slow.

5.5 The perceptron as a starting point

The perceptron is a single-layer classifier. For an input I and output a with desired label d , the perceptron update is

$$w_i \leftarrow w_i + r I_i(d - a), \quad (9)$$

where r is the learning rate. Backpropagation generalizes this delta-rule idea to multi-layer networks by computing how each weight contributes to the overall output error.

Algorithm 1 Perceptron training (outline)

- 1: Initialize weights randomly.
 - 2: **repeat**
 - 3: **for** each training example (I, d) **do**
 - 4: Compute output a .
 - 5: For each weight: $w_i \leftarrow w_i + r I_i(d - a)$.
 - 6: **end for**
 - 7: **until** all examples are correctly predicted or a stopping criterion is met.
-

5.6 Multi-layer feed-forward networks

A feed-forward network with one hidden layer can approximate any continuous function. One way to see this is by simulating a Fourier series with a one-hidden-layer network: any continuous function can be written as

$$f(x) = a_0 + \sum_{n=1}^{\infty} c_n \sin(nx + \theta_n), \quad (10)$$

where x is the input node, $f(x)$ the output node, $\sin(\cdot)$ the activation function, a_0 the bias of the output node, θ_n the bias of the n -th hidden node, n the weight of the corresponding input-hidden link, and c_n the weight from the n -th hidden node to the output.

Two hidden layers suffice to approximate any function, but the number of units grows exponentially with the number of inputs. There are several approaches to finding a good network structure; the most useful is the family of *hill-climbing* techniques, which modify the existing network structure dynamically. Multi-layer learning does not guarantee convergence to the global minimum, only to *some* minimum, possibly local. Standard heuristics for escaping a local minimum include injecting an artificial noise (called *jitter*) into the weight updates so the trajectory is kicked away from the trap, and *early stopping* to halt training before the network starts memorizing. These techniques are not derived in detail here.

The most thoroughly studied training method for multi-layer networks is backpropagation, derived in the next section.

6 Backpropagation

Backpropagation (the *generalized delta rule*; in numerical analysis the same algorithm appears under the name *heavy ball method*) is the most common supervised learning method for feed-forward neural networks. Application of the method involves two phases. In the first (forward) phase, signals from the inputs propagate through the network to the

outputs, where they are compared with the desired outputs. In the second (backward) phase, the error signal is passed backward from the output layer towards the input layer through the hidden layers, modifying weights according to each weight's contribution to the overall error. This is why the method is named *backpropagation*.

In a more geometric framing, backpropagation is gradient descent in weight space: the gradient is taken on the error surface that the current set of weights defines. It converges to a local minimum, with no guarantee of finding the global solution. Many techniques exist for dealing with this (jittering, early stopping, momentum; see below). The derivation below starts from first principles.

6.1 Error function

For a single training example, define the output error at unit i as

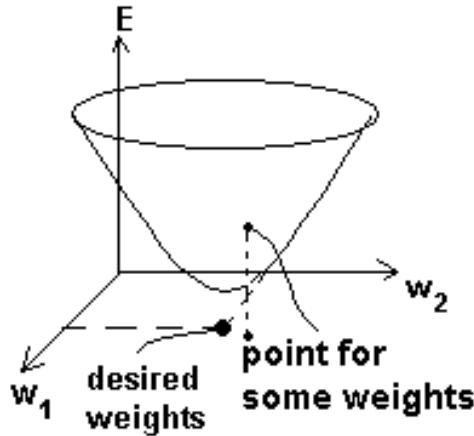
$$\varepsilon_i = d_i - a_i, \quad (11)$$

where d_i is the desired output and a_i is the actual output. To avoid sign cancellation, the squared error is commonly used:

$$E = \frac{1}{2} \sum_i \varepsilon_i^2 = \frac{1}{2} \sum_i (d_i - a_i)^2. \quad (12)$$

6.2 Gradient descent in weight space

Summing the per-example errors over all training cases makes E a function of all weights: $E = E(\mathbf{w})$. With $\|\mathbf{w}\|$ free parameters, this defines a hyper-surface in $(\|\mathbf{w}\| + 1)$ -dimensional space. Minimizing the error is the operation of moving the current point along the steepest descent direction on that surface, by adjusting all weights as parameters:



Let w_{ij} denote the weight from node j in the previous layer to node i in the current layer. Gradient descent updates parameters along the negative gradient:

$$\Delta w_{ij} = -r \frac{\partial E}{\partial w_{ij}}, \quad (13)$$

where r is the learning-rate parameter (a normalizing multiplier controlling the step size).

6.3 Derivation of the generalized delta rule

For a neuron i in some layer,

$$z_i = \sum_j w_{ij} a_j + b_i, \quad a_i = g(z_i), \quad (14)$$

where a_j are activations from the previous layer. By the chain rule,

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial a_i} \frac{\partial a_i}{\partial z_i} \frac{\partial z_i}{\partial w_{ij}} = \frac{\partial E}{\partial a_i} g'(z_i) a_j. \quad (15)$$

Hence,

$$\Delta w_{ij} = -r \left(\frac{\partial E}{\partial a_i} \right) g'(z_i) a_j. \quad (16)$$

Output layer. For an output unit i ,

$$\frac{\partial E}{\partial a_i} = \frac{\partial}{\partial a_i} \left(\frac{1}{2} (d_i - a_i)^2 \right) = -(d_i - a_i). \quad (17)$$

Substituting into (16) yields

$$\Delta w_{ij} = r a_j (d_i - a_i) g'(z_i). \quad (18)$$

Hidden layers. For a hidden unit i , the error depends on a_i only through units k in the next layer. Using the chain rule,

$$\frac{\partial E}{\partial a_i} = \sum_k \frac{\partial E}{\partial z_k} \frac{\partial z_k}{\partial a_i} = \sum_k \frac{\partial E}{\partial z_k} w_{ki}, \quad (19)$$

since $z_k = \sum_j w_{kj} a_j + b_k$ implies $\partial z_k / \partial a_i = w_{ki}$.

6.4 Local error signal (delta / beta)

Define the local error signal (often called *delta*; here denoted β) by

$$\beta_i \stackrel{\text{def}}{=} (d_i - a_i) g'(z_i) \quad (\text{output units}). \quad (20)$$

For hidden units,

$$\beta_i = g'(z_i) \sum_k w_{ki} \beta_k. \quad (21)$$

With this notation the update is local:

$$\Delta w_{ij} = r a_j \beta_i. \quad (22)$$

6.5 Sigmoid-specific form

When the activation is the sigmoid $g(z) = 1/(1 + e^{-z})$, the derivative simplifies in terms of the activation itself:

$$g'(z_i) = \frac{1}{1 + e^{-z_i}} \cdot \frac{e^{-z_i}}{1 + e^{-z_i}} = a_i(1 - a_i), \quad (23)$$

so the recursive formulas reduce to

$$\Delta w_{ij} = r a_i a_j (1 - a_i) \beta_i, \quad (24)$$

$$\beta_i = (d_i - a_i) a_i (1 - a_i) \quad (\text{output units}), \quad (25)$$

$$\beta_i = a_i (1 - a_i) \sum_k w_{ki} \beta_k \quad (\text{hidden units}). \quad (26)$$

These three equations appear constantly in the neural-network literature of this period.

6.6 Tensor (matrix) form

Let a be the vector of activations from the previous layer and β the vector of local error signals in the current layer. The component-wise rule $\Delta w_{ij} = r a_j \beta_i$ is the outer product

$$\Delta W = r \beta a^\top. \quad (27)$$

6.7 Backpropagation algorithm (outline)

Algorithm 2 Backpropagation training for a feed-forward network

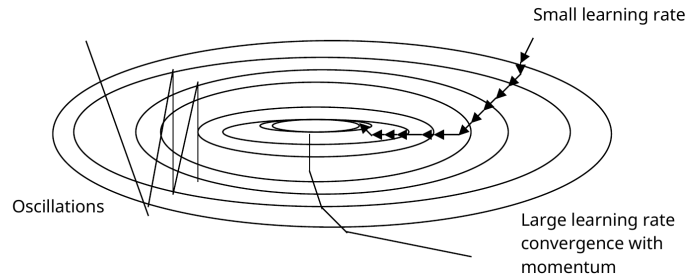
- 1: Choose a learning rate r and initialize weights/biases (small random values).
 - 2: **repeat**
 - 3: **for** each training example (x, d) **do**
 - 4: **Forward pass:** compute activations layer by layer.
 - 5: **Compute output error:** $\beta^{(L)} = (d - a^{(L)}) \odot g'(z^{(L)})$.
 - 6: **Backward pass:** for $\ell = L - 1, \dots, 1$ compute $\beta^{(\ell)} = g'(z^{(\ell)}) \odot (W^{(\ell+1)})^\top \beta^{(\ell+1)}$.
 - 7: **Update:** $W^{(\ell)} \leftarrow W^{(\ell)} + r \beta^{(\ell)} (a^{(\ell-1)})^\top$ (and similarly for biases).
 - 8: **end for**
 - 9: **until** a stopping criterion is met (error threshold, maximum epochs, or validation performance).
-

6.8 Learning rate and momentum

The learning rate r should be chosen as large as possible without driving the training into oscillation. A rate that is too small makes learning unbearably slow; a rate that is too large causes the error to oscillate or diverge and learning stops altogether. To stabilize the update without sacrificing speed, the weight change is often made dependent on the previous weight change via a *momentum* term:

$$\Delta w_{ij}(t + 1) = r \beta_i a_j + \alpha \Delta w_{ij}(t), \quad (28)$$

where $\alpha \in [0, 1)$ is the momentum coefficient. Training with a constant r alone is usually a tedious trial-and-error process.

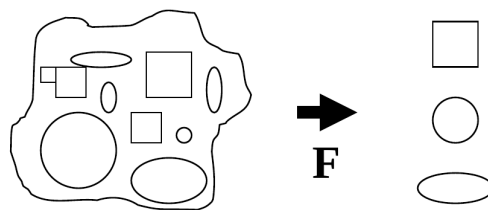


Convergence paths on a quadratic error surface (contour lines shown) for three learning-rate regimes: a small r inches along the gradient, a too-large r oscillates across the valley, and a large r combined with momentum cuts the fastest path to the minimum.

It is worth noting that standard backpropagation can be slow, especially with a fixed learning rate. Many modifications have been proposed to speed convergence, including Quickprop and RPROP. [2026: *Modern practice has converged on the Adam optimizer family (Kingma & Ba, 2014) and SGD with momentum, combined with explicit learning-rate schedules (warmup, cosine decay, step decay). Fixed learning rates are now rare outside of toy examples. Quickprop and RPROP are mostly of historical interest.*]

7 Applications

Pattern recognition is one of the most elaborated and successful domains for neural networks. Speech recognition, text and character recognition, handwriting recognition, vision tasks, and classification of radar/sonar signals are all examples. Marketing planning, stock-market prediction, and robotics sensing can also be formulated as pattern-recognition problems. Pattern classification can be viewed as a function mapping a domain of images (or feature vectors) to a finite set of output categories.



Pattern classification as a function F : an input image of mixed shapes is mapped to one of a finite set of output categories: here three (square, circle, ellipse).

Two concrete applications are described below: (i) classifying cells for cancer diagnosis and (ii) handwritten digit recognition. In both cases, the systems use backpropagation-trained feed-forward networks.

7.1 Classifying cells for cancer diagnosis

Source: Ciamac Moallemi (see Bibliography).

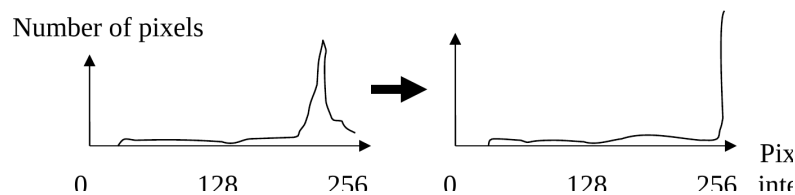
The goal of this work is to reduce routine manual microscopy by classifying cell images into “well” (useful) versus “not well” categories for cancer diagnosis. A key requirement is high recall of well cells: if a well cell is classified as not well, potentially valuable diagnostic information may be lost.

7.1.1 Data and processing pipeline

Images of size 256×240 pixels with 256 gray levels were used. Processing is divided into four steps:

1. thresholding,
2. segmentation,
3. feature extraction,
4. classification (performed by the neural network).

1) Thresholding. The goal is to separate objects from background. The image is partitioned into 16 equal segments (approximately 32×120 pixels each) in order to make the background more homogeneous within each segment. Background is then removed from each segment separately. Significant discontinuities in each histogram are removed, and the system selects an intensity threshold for the background. Pixels with gray-level values above this threshold are treated as background and assigned white.



Pixel-intensity histogram before and after thresholding. The background peak (left) collapses onto the white bin (right), leaving the foreground pixels separated from background.

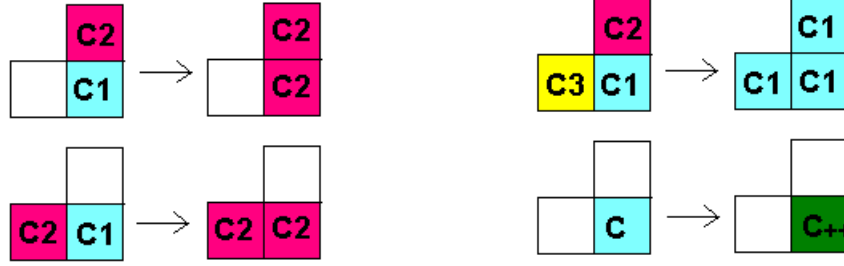
2) Segmentation. Objects are separated using a “blob coloring” algorithm (see Ballard & Brown). The algorithm sweeps an L-shaped template across the image and assigns a distinct color to each connected object. The method can be stated as:



L-shaped three-pixel template used by the scan: x_c is the current pixel, x_u its upper neighbor, and x_l its left neighbor.

Algorithm 3 Blob-coloring segmentation (outline)

```
1: Let the color counter  $k \leftarrow 1$ .
2: Define  $f(x) = 0$  if  $x$  is a background pixel, and  $f(x) = 1$  otherwise.
3: while there remain unprocessed pixels do
4:   Scan the image from top to bottom and left to right until  $f(x_c) = 1$ .
5:   if  $f(x_u) = 1$  and  $f(x_l) = 0$  then
6:      $color(x_c) \leftarrow color(x_u)$ 
7:   end if
8:   if  $f(x_l) = 1$  and  $f(x_u) = 0$  then
9:      $color(x_c) \leftarrow color(x_l)$ 
10:  end if
11:  if  $f(x_l) = 1$  and  $f(x_u) = 1$  then
12:    Merge labels:  $color(x_u) \leftarrow color(x_c)$  and  $color(x_l) \leftarrow color(x_c)$ 
13:  end if
14:  if  $f(x_l) = 0$  and  $f(x_u) = 0$  then
15:     $color(x_c) \leftarrow k$ ;  $k \leftarrow k + 1$ 
16:  end if
17: end while
```



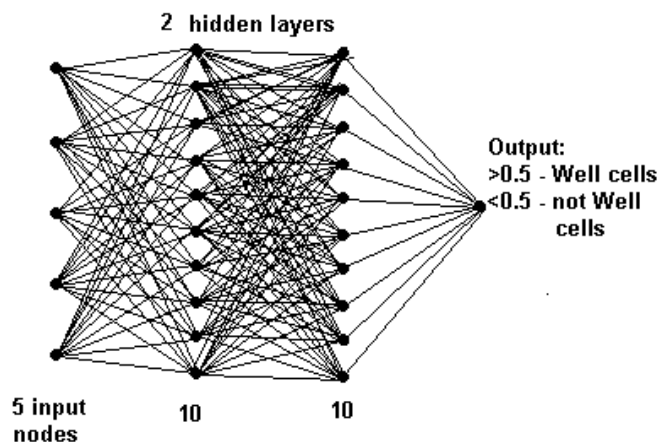
Blob-coloring transitions: assigning, propagating, merging, and starting a new label.

After this step, objects larger than 20 pixels can be detected reliably by their assigned labels.

3) Feature extraction. Features are numerical descriptors of each segmented object. Five features are used:

1. object area (typically 100–400 pixels),
2. circularity (ratio of object area to enclosing rectangle; typically 0.6–0.85),
3. nuclear area (nucleus segmented by thresholding with constant 0.9; typically < 150 pixels but > 20),
4. nuclear circularity (typically above 0.35),
5. ratio of nuclear area to total cell area (typically below 0.85).

4) Classification by neural network. The extracted features form the neural-network input. The fully connected network has 5 input nodes, two hidden layers of 10 nodes each, and 1 output node. An output value greater than 0.5 signifies a “well” cell.



Cancer-cell classifier: 5 inputs $\rightarrow$ 10 $\rightarrow$ 10 $\rightarrow$ 1 output; > 0.5 signifies a well cell.

The architecture was selected experimentally: smaller networks did not adapt well to the training set, while larger networks trained too slowly and generalized poorly. Training used Quickprop (a variant of backpropagation). Quickprop updates each weight using the gradient at two consecutive steps:

$$\Delta w(t) = \frac{\partial E / \partial w(t)}{\partial E / \partial w(t-1) - \partial E / \partial w(t)} \cdot \Delta w(t-1). \quad (29)$$

7.1.2 Results

A previously developed “selective mapping” algorithm based on a tree classifier achieved a misclassification level of 23.1%, which is not acceptable for clinical use. The neural-network classifier reduced the error rates to 6.4% for well cells and 3% for not-well cells, with an overall error rate of 3.5%. Computation was fast: approximately three seconds per image on a Sun 3/80, benefiting from the parallel structure of the computations.

7.2 Handwritten digit recognition using constrained automatic learning

Source: LeCun et al. (see Bibliography).

This work addressed recognition of handwritten zip codes on postal envelopes. Digits may be incomplete, blotchy, or of poor quality. Classical pixel-based classifiers often perform poorly because they do not incorporate prior knowledge about the problem. Experiments showed that a large unconstrained network can be difficult to train on real databases with good generalization. Performance improved when domain knowledge was incorporated into the network design.

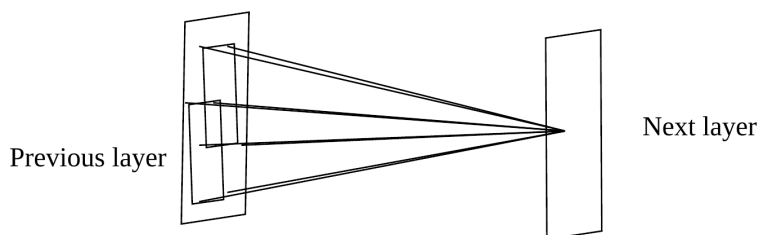
7.2.1 Preprocessing

Preprocessing consisted primarily of normalizing digit shape and skew so that inputs fit within a 32×32 or 16×16 pixel window.

7.2.2 Shared weights and feature maps

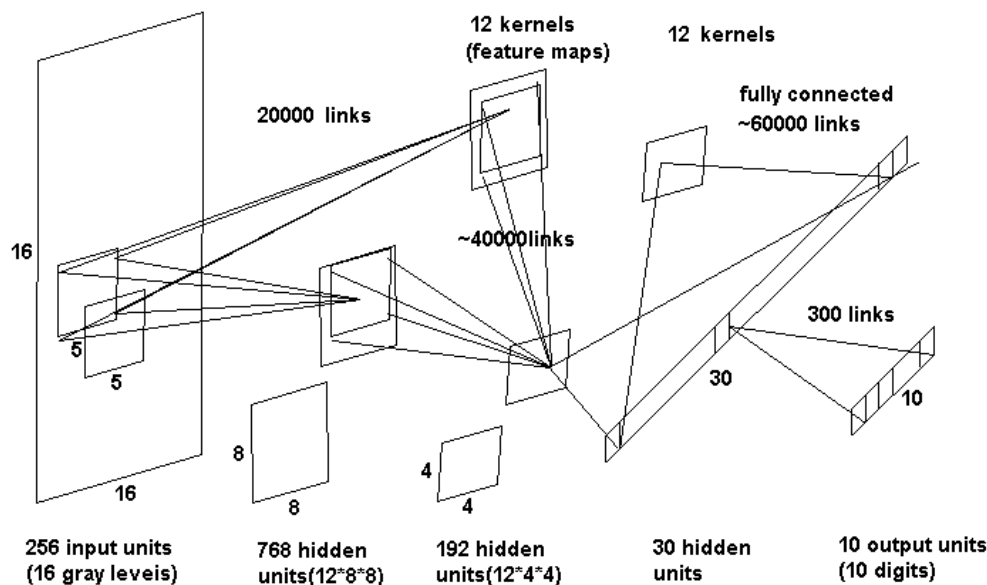
A key idea is to incorporate the two-dimensional geometric nature of images and to build shift invariance into the classifier. Shared weights in early layers greatly reduce the number of free parameters and improve generalization. The network implements *feature maps*: within each map, all neurons apply the same weights to different spatial locations in the input. Thus, all units in a map detect the same feature at different locations, providing translation (shift) invariance. A standard fully connected network, in contrast, would treat a translated digit as a completely new pattern.

Connections therefore act as feature detectors. The first hidden layer can detect multiple distinct local features, each of which can occur at different locations.



Receptive-field connection between layers: a single unit in the next layer receives inputs only from a local patch of the previous layer. Within a feature map the same weights are reused at every patch position, so the same detector applies at every location in the input.

7.2.3 Network architecture and training



LeCun network: 256 input units (16×16) $\rightarrow$ 768 hidden units (12 feature maps of 8×8) $\rightarrow$ 192 hidden units (12 maps of 4×4) $\rightarrow$ 30 hidden units $\rightarrow$ 10 output units. The first two hidden layers perform convolutions with shared weights; the last two are fully connected.

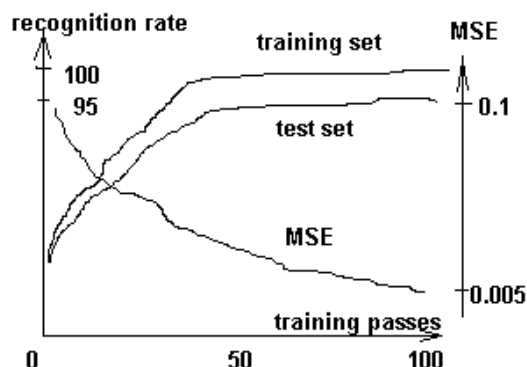
Connections within a given feature map are constrained to share weights. For example, all 64 units in a kernel (feature map) in the first hidden layer use the same set of 25 weights. Because of weight sharing, the network has 9,760 weights, substantially fewer than the number of potential links. The first two hidden layers perform convolutions,

while the third and fourth (output) layers are fully connected and perform the final classification. [2026: *This shared-weight feature-map design is now recognized as the origin of the convolutional neural network (CNN) family that dominated image recognition from AlexNet (2012) onward.*]

Weights were initialized randomly in the range $[-2.4/f_i, 2.4/f_i]$, where f_i is the number of inputs (fan-in) to the unit associated with the connection. Mean squared error was used as the cost function. [2026: *Modern classification networks use cross-entropy loss (with a softmax output layer) rather than MSE. Cross-entropy gives stronger gradients when the prediction is far from the target, so training converges faster and more reliably. MSE is still the standard choice for regression problems.*] Training was performed in an on-line (stochastic gradient) fashion: patterns were presented in a constant order for 28 passes, and weights were updated after each individual pattern. This procedure often converges faster on highly redundant data. The network was trained on 7,300 examples and tested on a separate set of 2,000 examples. The learning rate was fixed throughout training.

7.2.4 Performance and impact

Performance reached approximately 99%, and the system was implemented in custom VLSI for high-speed postal sorting, achieving throughput from camera to classified digit of more than 10 digits per second. This work was among the first to demonstrate that a general-purpose neural-network chip could serve as an accelerator in a larger system, enabling real problems with strong regularities to be solved effectively by neural networks.



Approximate behavior of mean squared error (and recognition rate) as a function of passes through the training set.

8 Summary

Neural networks are useful when either no explicit algorithm is known for a problem, or the number of interacting variables makes hand-crafted rules impractical. Traditional application fields include pattern recognition (including signature analysis as a sub-field), speech and vision recognition, sound and vibration analysis, process control, marketing planning, stock-market prediction, and robotics / sensing.

Feed-forward networks form roughly **90% of successful neural-network applications** (as of 1997). [2026: *This share was accurate in 1997 but no longer holds. The post-2012 deep-learning era is dominated by convolutional networks for vision, recurrent and transformer networks for sequences and language, and large generative models. Pure*

feed-forward networks remain the foundational building block (every modern architecture composes from them), but they are no longer the dominant deployed form.] With enough resources and training data, almost any function can be approximated to arbitrary precision by a sufficiently large feed-forward network. Backpropagation is by far the most common training method for these networks; it is slow in a naive implementation, and many variants (Quickprop, RPROP, momentum, jittering, early stopping) have been proposed to speed convergence or to escape local minima.

When a network has too many trainable weights it may become erratic and unreliable due to overfitting; a practical rule of thumb is that the number of trainable weights influencing any output should be smaller than the number of training samples. At the same time, an overly small network cannot learn the required mapping. Tuning the learning rate further complicates the trade-off.

The single most effective lever is heuristics: incorporating problem-specific knowledge into the network design. Two illustrations from the applications:

- Two-dimensional geometry and shift invariance built into the topology of the LeCun handwritten-digit network.
- Sequential hidden layers performing feature extraction at successive abstraction levels, as in both applications above.

Without such heuristics, even a correctly-trained network often fails to generalize. The thesis of this report is that the architecture and preprocessing pipeline carry as much of the work as the gradient-descent algorithm itself; they are necessary, not cosmetic.

9 Bibliography

1. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, 323(6088), October 1986, pp. 533–536.
2. Ben J. A. Kröse and Patrick van der Smagt, *An Introduction to Neural Networks*, 4th edition, September 1991.
3. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Prentice Hall, 1995, pp. 563–591.
4. Y. LeCun et al., “Handwritten Digit Recognition: Applications of Neural Network Chips and Automatic Learning,” *IEEE Communications Magazine*, November 1989, 27(11), pp. 41–46.
5. Ciamac Moallemi, “Classifying Cells for Cancer Diagnosis Using Neural Networks,” *IEEE Expert*, December 1991, pp. 8–12.
6. G. Dreyfus, “Neural Networks for Automatic Recognition of Handwritten Digits,” in *Applications of Neural Networks*, edited by H. G. Schuster, VCH.
7. D. H. Ballard and C. M. Brown, *Computer Vision*, Prentice Hall, Englewood Cliffs, NJ, 1982.

8. Online resources (as originally consulted):

<ftp://ftp.sas.com/pub/neural/>

<http://www.shef.ac.uk/psychology/gurney/notes/contents.html>

<http://www.cs.stir.ac.uk/~lss/NNIntro/InvSlides.html>

<http://www.mcs.com/~drt/bprefs.html>